ASSIGNMENT 8

Basic SQL Queries and Aggregate Functions

WHERE, ORDER BY, GROUP BY, HAVING, COUNT, SUM, AVG, MIN, MAX – with real outputs

1. Objective

This assignment demonstrates the fundamental SQL clauses for retrieving and manipulating data: filtering with **WHERE**, sorting with **ORDER BY**, grouping with **GROUP BY**, filtering groups with **HAVING**, and the aggregate functions **COUNT, SUM, AVG, MIN** and **MAX**. Each query is followed by its output and a short explanation.

2. Retrieving and Filtering Data (WHERE)

Query 1 – Simple condition: products costing more than INR 5,000

SQL query

```
SELECT product_id, product_name, brand, price
FROM Products
WHERE price > 5000;
```

Output

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	product_name	brand	price
1	Redmi Note 13 5G (8GB/128GB)	Xiaomi	17999.00
17	Digital Air Fryer 4.1L	Philips	7999.00
NULL	NULL	NULL	NULL

Explanation: WHERE keeps only rows for which the condition is true; here the expensive products.

Query 2 – Combining conditions with AND / OR

SQL query

```
SELECT product_name, price, stock_quantity
FROM Products
WHERE category_id = 1 AND (price < 1500 OR stock_quantity > 100);
```

Output

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	price	stock_quantity
Rockerz 450 Bluetooth Headphones	1499.00	120
25W Fast Charger	1199.00	200
M331 Silent Plus Wireless Mouse	899.00	85

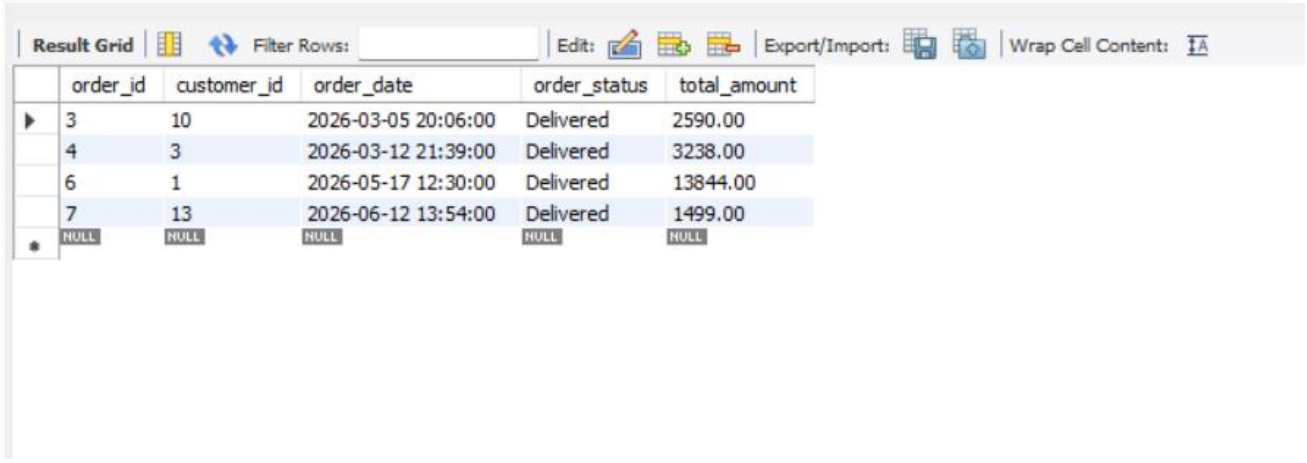
Explanation: Electronics (category 1) that are either cheap or well stocked. Brackets make the OR apply before the AND.

Query 3 – Range and list conditions: BETWEEN and IN

SQL query

```
SELECT order_id, customer_id, order_date, order_status, total_amount
FROM Orders
WHERE order_date BETWEEN '2026-03-01' AND '2026-06-30 23:59:59'
  AND order_status IN ('Delivered', 'Shipped');
```

Output



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

order_id	customer_id	order_date	order_status	total_amount
3	10	2026-03-05 20:06:00	Delivered	2590.00
4	3	2026-03-12 21:39:00	Delivered	3238.00
6	1	2026-05-17 12:30:00	Delivered	13844.00
7	13	2026-06-12 13:54:00	Delivered	1499.00
NULL	NULL	NULL	NULL	NULL

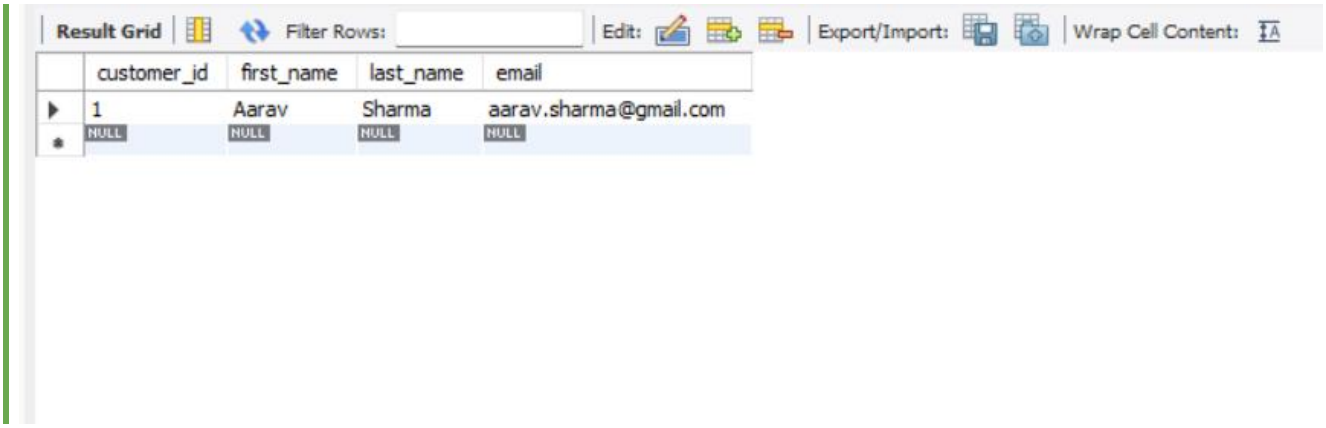
Explanation: BETWEEN selects a date range and IN matches any value from a list – here the delivered or shipped orders placed from March to June 2026.

Query 4 – Pattern matching with LIKE

SQL query

```
SELECT customer_id, first_name, last_name, email
FROM Customers
WHERE email LIKE '%@gmail.com' AND first_name LIKE 'A%';
```

Output



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	first_name	last_name	email
1	Aarav	Sharma	aarav.sharma@gmail.com
NULL	NULL	NULL	NULL

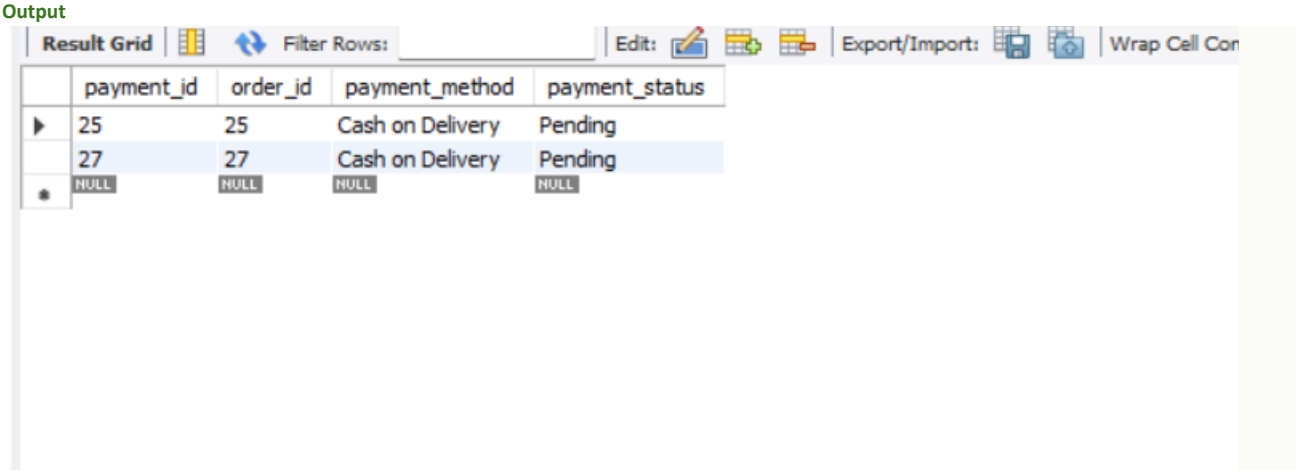
Explanation: % stands for any sequence of characters: customers whose first name starts with A and who use a Gmail address.

Query 5 – Testing for NULL

SQL query

```
SELECT payment_id, order_id, payment_method, payment_status
FROM Payments
WHERE payment_date IS NULL;
```

Output



payment_id	order_id	payment_method	payment_status
25	25	Cash on Delivery	Pending
27	27	Cash on Delivery	Pending
NULL	NULL	NULL	NULL

Explanation: IS NULL finds payments that have no payment date yet – these are the Cash on Delivery payments still pending.

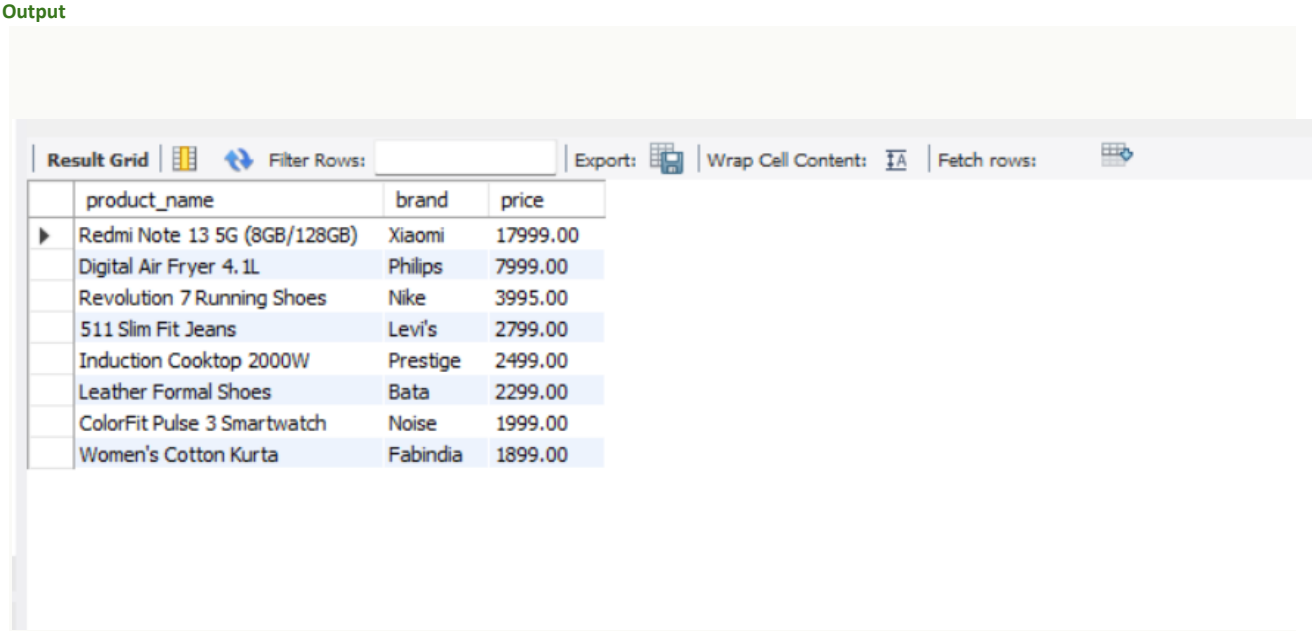
3. Sorting Records (ORDER BY)

Query 6 – Sort by price, highest first

SQL query

```
SELECT product_name, brand, price
FROM Products
ORDER BY price DESC
LIMIT 8;
```

Output



product_name	brand	price
Redmi Note 13 5G (8GB/128GB)	Xiaomi	17999.00
Digital Air Fryer 4.1L	Philips	7999.00
Revolution 7 Running Shoes	Nike	3995.00
511 Slim Fit Jeans	Levi's	2799.00
Induction Cooktop 2000W	Prestige	2499.00
Leather Formal Shoes	Bata	2299.00
ColorFit Pulse 3 Smartwatch	Noise	1999.00
Women's Cotton Kurta	Fabindia	1899.00

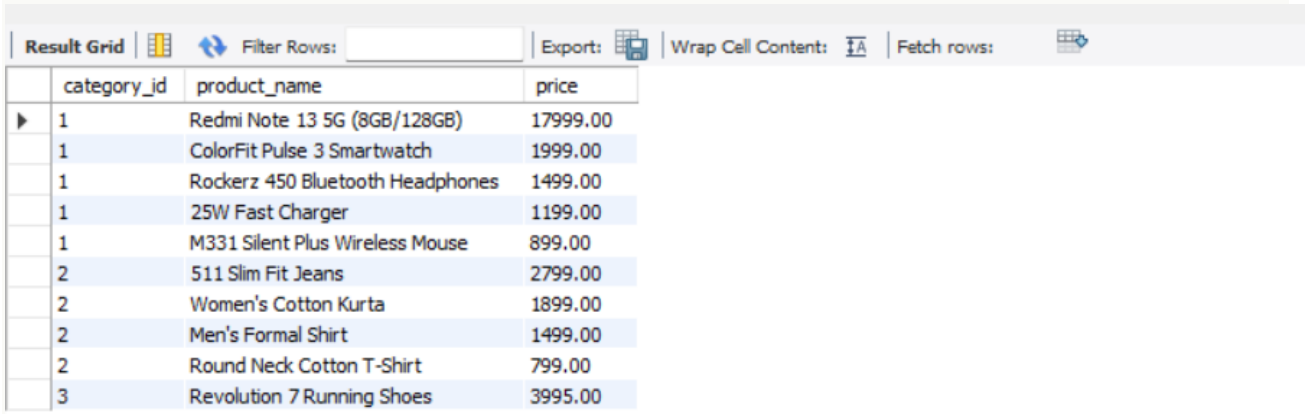
Explanation: ORDER BY ... DESC sorts from largest to smallest; LIMIT 8 keeps only the top eight rows.

Query 7 – Sort by several columns

SQL query

```
SELECT category_id, product_name, price
FROM Products
ORDER BY category_id ASC, price DESC
LIMIT 10;
```

Output



category_id	product_name	price
1	Redmi Note 13 5G (8GB/128GB)	17999.00
1	ColorFit Pulse 3 Smartwatch	1999.00
1	Rockerz 450 Bluetooth Headphones	1499.00
1	25W Fast Charger	1199.00
1	M331 Silent Plus Wireless Mouse	899.00
2	511 Slim Fit Jeans	2799.00
2	Women's Cotton Kurta	1899.00
2	Men's Formal Shirt	1499.00
2	Round Neck Cotton T-Shirt	799.00
3	Revolution 7 Running Shoes	3995.00

Explanation: Rows are ordered by category first; within each category the most expensive product comes first.

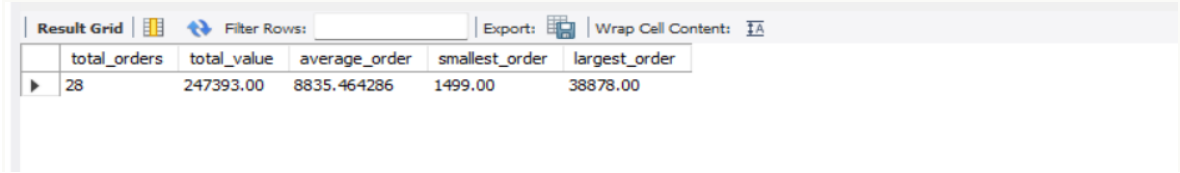
4. Aggregate Functions

Query 8 – COUNT, SUM, AVG, MIN and MAX over the whole Orders table

SQL query

```
SELECT COUNT(*)          AS total_orders,
       SUM(total_amount) AS total_value,
       AVG(total_amount) AS average_order,
       MIN(total_amount) AS smallest_order,
       MAX(total_amount) AS largest_order
FROM Orders;
```

Output



total_orders	total_value	average_order	smallest_order	largest_order
28	247393.00	8835.464286	1499.00	38878.00

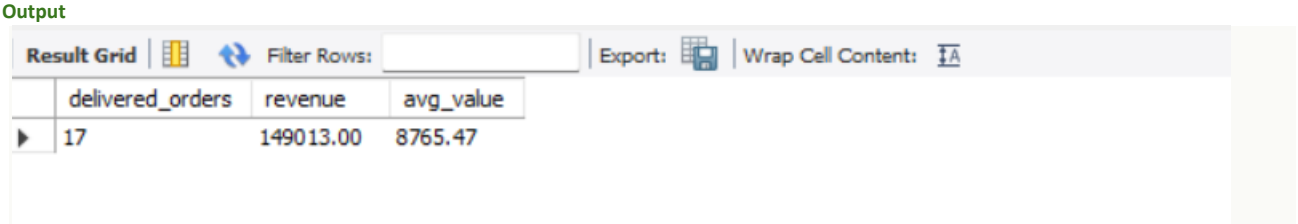
Explanation: Without GROUP BY, an aggregate query collapses all rows into one summary row describing the entire table.

Query 9 – Aggregates with a WHERE filter (only delivered orders)

SQL query

```
SELECT COUNT(*) AS delivered_orders, SUM(total_amount) AS revenue, ROUND(AVG(total_amount),2) AS avg_value
FROM Orders
WHERE order_status = 'Delivered';
```

Output



delivered_orders	revenue	avg_value
17	149013.00	8765.47

Explanation: WHERE is applied before aggregation, so only delivered orders contribute to the count, revenue and average.

Query 10 – COUNT(DISTINCT ...)

SQL query

```
SELECT COUNT(DISTINCT customer_id) AS customers_who_ordered, COUNT(*) AS orders FROM Orders;
```

Output

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customers_who_ordered	orders
13	28

Explanation: DISTINCT counts each customer once, regardless of how many orders they placed. Customers who never ordered are not counted.

5. Grouping Records (GROUP BY)

Query 11 – Number of products and price statistics per category

SQL query

```
SELECT category_id, COUNT(*) AS products, MIN(price) AS cheapest,
       MAX(price) AS dearest, ROUND(AVG(price),2) AS avg_price
FROM Products
GROUP BY category_id
ORDER BY category_id;
```

Output

Result Grid | Filter Rows: | Export: | Wrap Cell C

category_id	products	cheapest	dearest	avg_price
1	5	899.00	17999.00	4719.00
2	4	799.00	2799.00	1749.00
3	4	1499.00	3995.00	2398.00
4	4	699.00	7999.00	2999.00
5	4	299.00	899.00	511.50
6	4	549.00	1299.00	774.00

Explanation: GROUP BY creates one group per category, and each aggregate is calculated inside its group.

Query 12 – Orders and revenue by order status

SQL query

```
SELECT order_status, COUNT(*) AS orders, SUM(total_amount) AS total_value
FROM Orders
GROUP BY order_status
ORDER BY total_value DESC;
```

Output

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_status	orders	total_value
Delivered	17	149013.00
Placed	4	36083.00
Shipped	4	32786.00
Cancelled	3	29511.00

Explanation: Shows how much business sits in each stage of the order life-cycle.

Query 13 – Payments by method (successful only)

SQL query

```
SELECT payment_method, COUNT(*) AS payments, SUM(amount_paid) AS amount_collected
FROM Payments
WHERE payment_status = 'Successful'
GROUP BY payment_method
ORDER BY amount_collected DESC;
```

Output

```
+------------------+----------+------------------+
| payment_method   | payments | amount_collected |
+------------------+----------+------------------+
| UPI              |       11 |        114279.00 |
| Card             |        4 |         39155.00 |
| Net Banking      |        5 |         30223.00 |
| Cash on Delivery |        3 |         25732.00 |
+------------------+----------+------------------+
```

Explanation: WHERE filters the rows first, GROUP BY then groups what is left – a typical business report on preferred payment methods.

Query 14 – Monthly sales trend

SQL query

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, COUNT(*) AS orders, SUM(total_amount) AS sales
FROM Orders
WHERE order_status <> 'Cancelled'
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

Output

```
+---------+--------+----------+
| month   | orders | sales    |
+---------+--------+----------+
| 2026-01 |      1 | 24379.00 |
| 2026-02 |      1 |  4591.00 |
| 2026-03 |      2 |  5828.00 |
| 2026-05 |      1 | 13844.00 |
| 2026-06 |      1 |  1499.00 |
| 2026-07 |      4 | 69355.00 |
| 2026-08 |      3 | 10188.00 |
| 2026-09 |      4 | 19329.00 |
| 2026-10 |      8 | 68869.00 |
+---------+--------+----------+
```

Explanation: Grouping on a date expression gives month-wise figures; cancelled orders are excluded.

6. Filtering Groups (HAVING)

Query 15 – Customers who placed more than one order

SQL query

```
SELECT customer_id, COUNT(*) AS orders, SUM(total_amount) AS total_spent
FROM Orders
GROUP BY customer_id
HAVING COUNT(*) > 1
ORDER BY total_spent DESC;
```

Output

```
+-------------+--------+-------------+
| customer_id | orders | total_spent |
+-------------+--------+-------------+
|          14 |      4 |    52611.00 |
|           3 |      4 |    36723.00 |
|           7 |      4 |    35833.00 |
|          10 |      3 |    31560.00 |
|           8 |      2 |    25595.00 |
|          12 |      2 |    11259.00 |
|          13 |      2 |     7493.00 |
|           9 |      2 |     7391.00 |
+-------------+--------+-------------+
```

Explanation: HAVING filters *groups* after aggregation (repeat customers), whereas WHERE filters individual rows before aggregation.

Query 16 – Categories whose average product price exceeds INR 1,500

SQL query

```
SELECT category_id, ROUND(AVG(price),2) AS avg_price, COUNT(*) AS products
FROM Products
GROUP BY category_id
HAVING AVG(price) > 1500;
```

Output

```
+-------------+-----------+----------+
| category_id | avg_price | products |
+-------------+-----------+----------+
|           1 |   4719.00 |        5 |
|           2 |   1749.00 |        4 |
|           3 |   2398.00 |        4 |
|           4 |   2999.00 |        4 |
+-------------+-----------+----------+
```

Explanation: Only categories whose group average passes the threshold are returned.

Query 17 – Products with at least two reviews and their average rating

SQL query

```
SELECT product_id, COUNT(*) AS reviews, ROUND(AVG(rating),2) AS avg_rating,
       MIN(rating) AS lowest, MAX(rating) AS highest
FROM Reviews
GROUP BY product_id
HAVING COUNT(*) >= 2
ORDER BY avg_rating DESC;
```

Output

```
+------------+---------+------------+--------+---------+
| product_id | reviews | avg_rating | lowest | highest |
+------------+---------+------------+--------+---------+
|         13 |       2 |       5.00 |      5 |       5 |
|          1 |       3 |       4.67 |      4 |       5 |
|          2 |       2 |       4.50 |      4 |       5 |
|          7 |       2 |       4.00 |      4 |       4 |
|         10 |       3 |       3.67 |      3 |       4 |
|         24 |       2 |       3.50 |      2 |       5 |
+------------+---------+------------+--------+---------+
```

Explanation: Combines GROUP BY, HAVING and all five aggregate functions to summarise customer feedback.

Query 18 – WHERE and HAVING together: best-selling products among non-cancelled orders

SQL query

```
SELECT oi.product_id, SUM(oi.quantity) AS units_sold, SUM(oi.quantity * oi.unit_price) AS revenue
FROM Order_Items oi JOIN Orders o ON o.order_id = oi.order_id
WHERE o.order_status <> 'Cancelled'
GROUP BY oi.product_id
HAVING SUM(oi.quantity) >= 3
ORDER BY units_sold DESC, revenue DESC;
```

Output

```
+------------+------------+-----------+
| product_id | units_sold | revenue   |
+------------+------------+-----------+
|          4 |          7 |   6437.00 |
|         21 |          7 |   2093.00 |
|          1 |          6 | 113754.00 |
|          2 |          6 |   9234.00 |
|         24 |          6 |   3294.00 |
|         10 |          5 |  19975.00 |
|          7 |          4 |   5996.00 |
|         25 |          4 |   5196.00 |
|          3 |          4 |   5084.00 |
|         18 |          4 |   1996.00 |
|         14 |          3 |   7497.00 |
|          5 |          3 |   5997.00 |
|         13 |          3 |   4497.00 |
|          8 |          3 |   2397.00 |
|         15 |          3 |   2397.00 |
|         23 |          3 |   1797.00 |
+------------+------------+-----------+
```

Explanation: WHERE removes cancelled orders row by row; HAVING keeps only products that sold at least 3 units in total.

7. Manipulating Data (INSERT, UPDATE, DELETE)

The following statements modify data. Each is run inside a transaction and rolled back so that the data set used in later assignments stays unchanged; a SELECT inside the transaction shows the effect.

Query 19 – UPDATE: apply a 10% discount to all Footwear products

SQL query

```
START TRANSACTION;
SELECT product_id, LEFT(product_name,28) AS product, price FROM Products WHERE category_id = 3;
UPDATE Products SET price = ROUND(price * 0.90, 2) WHERE category_id = 3;
SELECT product_id, LEFT(product_name,28) AS product, price FROM Products WHERE category_id = 3;
ROLLBACK;
```

Output

```
+------------+----------------------------+---------+
| product_id | product                    | price   |
+------------+----------------------------+---------+
|         10 | Revolution 7 Running Shoes | 3995.00 |
|         11 | Leather Formal Shoes       | 2299.00 |
|         12 | Adilette Comfort Slides    | 1799.00 |
|         13 | Casual Canvas Sneakers     | 1499.00 |
+------------+----------------------------+---------+
+------------+----------------------------+---------+
| product_id | product                    | price   |
+------------+----------------------------+---------+
|         10 | Revolution 7 Running Shoes | 3595.50 |
|         11 | Leather Formal Shoes       | 2069.10 |
|         12 | Adilette Comfort Slides    | 1619.10 |
|         13 | Casual Canvas Sneakers     | 1349.10 |
+------------+----------------------------+---------+
```

Explanation: The first SELECT shows the old prices, the second shows the discounted prices; ROLLBACK then restores the original data.

Query 20 – INSERT a new product and DELETE it again

SQL query

```
START TRANSACTION;
INSERT INTO Products (category_id, product_name, description, price, brand)
VALUES (4, 'Electric Kettle 1.5L', 'Stainless steel kettle', 1299.00, 'Prestige');
SELECT product_id, product_name, price, stock_quantity FROM Products WHERE product_name = 'Electric Kettle 1.5L';
DELETE FROM Products WHERE product_name = 'Electric Kettle 1.5L';
SELECT COUNT(*) AS kettles_left FROM Products WHERE product_name = 'Electric Kettle 1.5L';
ROLLBACK;
```

Output

```
+------------+---------------------+---------+----------------+
| product_id | product_name        | price   | stock_quantity |
+------------+---------------------+---------+----------------+
|         26 | Electric Kettle 1.5L | 1299.00 |              0 |
+------------+---------------------+---------+----------------+
+--------------+
| kettles_left |
+--------------+
|            0 |
+--------------+
```

Explanation: The INSERT omits stock_quantity, so the DEFAULT 0 from Assignment 6 is applied. After the DELETE no kettle remains.

Query 21 – UPDATE with a condition: mark a shipment as delivered

SQL query

```
START TRANSACTION;
UPDATE Shipments SET shipment_status = 'Delivered' WHERE shipment_status = 'Delayed';
SELECT shipment_id, order_id, courier_name, shipment_status FROM Shipments WHERE order_id = 21;
ROLLBACK;
```

Output

```
+-------------+----------+--------------+-----------------+
| shipment_id | order_id | courier_name | shipment_status |
+-------------+----------+--------------+-----------------+
|          18 |       21 | Blue Dart    | Delivered       |
+-------------+----------+--------------+-----------------+
```

Explanation: The WHERE clause limits the change to delayed shipments; without it every shipment would be altered.

8. Conclusion

The queries show how WHERE selects rows, ORDER BY arranges them, GROUP BY summarises them into groups, HAVING filters those groups, and COUNT, SUM, AVG, MIN and MAX produce the statistics a store manager needs – order values, revenue by status and payment method, monthly sales, repeat customers and product ratings.